SISTEM PEMINJAMAN FASILITAS KAMPUS

APLIKASI BERIORENTASI SERVICE

Pengembangan REST API dengan Layanan Katalog, Layanan Peminjaman, dan Autentikasi JWT

KELOMPOK 6:

1. MARCHELLO ANDREAS LENGKONG	240211060045
2. MARCHO ALEANDRO CRISTIAN RORING	240211060072
3. MIRACLE AARON MARTHEN SUNDAH	240211060074
4. REHAN IBRAHIM	240211060078

UNIVERSITAS SAM RATULANGI

JURUSAN TEKNIK ELEKTRO

PROGRAM STUDI TEKNIK INFORMATIKA

2026

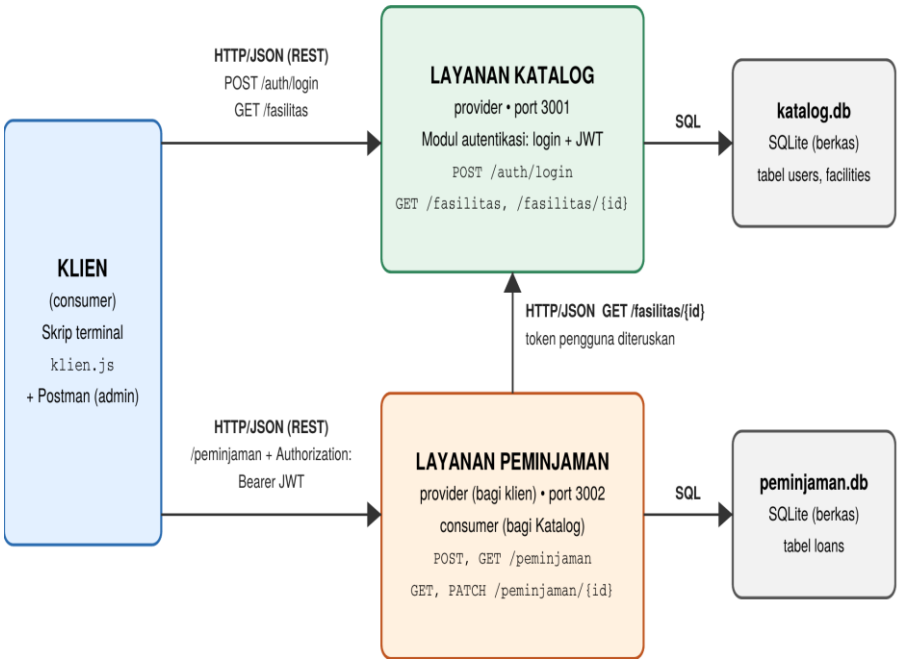
1. Pendahuluan dan Kebutuhan Sistem

Peminjaman ruangan dan peralatan kampus yang dicatat secara manual (formulir kertas atau pesan pribadi) mudah menimbulkan jadwal ganda, status pengajuan yang tidak transparan, dan beban kerja admin yang besar. Dampaknya kegiatan dapat batal mendadak dan fasilitas tidak terpakai optimal. Tujuan sistem ini adalah prototipe *backend* berbasis layanan: mahasiswa melihat katalog dan mengajukan peminjaman, admin memproses pengajuan, dengan jadwal yang tidak bertabrakan dan akses yang dibatasi peran serta kepemilikan data.

Mahasiswa (melihat fasilitas, mengajukan peminjaman, melihat pengajuan miliknya) dan *Admin* (melihat seluruh pengajuan, menyetujui atau menolak). Di dalam lingkup: tujuh endpoint, JWT, otorisasi peran dan kepemilikan, klien terminal. Di luar lingkup: registrasi, pembatalan, pengembalian, penambahan atau perubahan katalog, antarmuka grafis, *refresh token*, dan *deployment* publik.

Use case	**Alur berhasil**	**Alur gagal**
UC1 Melihat fasilitas (mahasiswa, admin)	Pengguna login dan memperoleh JWT, lalu GET /fasilitas dan GET /fasilitas/{id}. Sistem membalas 200 berisi data fasilitas.	Permintaan tanpa token atau dengan token kedaluwarsa dijawab 401 dan tidak ada data yang dikembalikan.
UC2 Mengajukan peminjaman (mahasiswa)	Mahasiswa mengirim POST /peminjaman (fasilitas, waktu mulai dan selesai, tujuan). Sistem memeriksa token, memanggil Katalog, memeriksa jadwal, lalu menyimpan pengajuan berstatus *pending* (201).	Jadwal bertumpang tindih dengan pengajuan *pending*/*approved* pada fasilitas yang sama dijawab 409 dan tidak ada pengajuan baru yang tersimpan.
UC3 Memproses pengajuan (admin)	Admin mengirim PATCH /peminjaman/{id} dengan status *approved* atau *rejected* untuk pengajuan *pending*. Sistem membalas 200 dan pemilik dapat melihat status barunya.	Mahasiswa yang mencoba menyetujui pengajuan dijawab 403 dan status pengajuan tidak berubah.

2. Arsitektur dan Konsep Layanan



Gambar 1. Diagram komponen. Panah menunjuk dari consumer ke provider; semua komunikasi memakai HTTP/JSON, dan tiap layanan memiliki berkas SQLite sendiri.

Layanan Katalog (port 3001) menyajikan daftar dan detail fasilitas dari data awal, serta memuat modul autentikasi yang menyimpan akun (password ter-*hash* bcrypt) dan menerbitkan JWT. *Layanan Peminjaman* (port 3002) mengelola pengajuan, validasi waktu, aturan bentrok, dan status; layanan ini memverifikasi token sendiri dan memanggil Katalog lewat HTTP. *Klien* adalah skrip terminal klien.js (login, katalog, pengajuan, daftar pengajuan sendiri); operasi admin didemonstrasikan lewat Postman.

Urutan pengajuan sebagai berikut: (1) *Pemeriksaan token*: middleware memverifikasi signature dan exp, memastikan peran mahasiswa, lalu memvalidasi input (400). (2) *Pemanggilan Katalog*: GET /fasilitas/{id} dengan token pengguna diteruskan; fasilitas tidak ada (404) atau nonaktif (409) menghentikan proses. (3) *Pemeriksaan jadwal*: mencari pengajuan *pending*/*approved* pada fasilitas yang sama dengan start_baru < end_lama dan end_baru > start_lama. (4) *Penyimpanan*: pengajuan *pending* disimpan; langkah 3 dan 4 berada dalam satu transaksi sinkron agar dua permintaan bersamaan tidak sama-sama lolos.

Empat prinsip inti layanan dan buktinya

Prinsip	Bukti pada proyek
Service contract terstandarisasi	Kontrak (KONTRAK_API.md, Panduan API) menetapkan prefiks /api/v1, nama resource, format JSON, format waktu UTC, kode status, dan satu format error {error:{status,code,message}} yang sama di kedua layanan. Modul jwtAuth.js identik di Katalog dan Peminjaman.
Loose coupling	Peminjaman hanya mengenal alamat KATALOG_URL dan memakai field id dan is_active; ia tidak membaca berkas, tabel, atau modul Katalog. Tabel loans menyimpan facility_id tanpa *foreign key*. Saat Katalog dihentikan, POST /peminjaman dijawab 503 sedangkan GET /peminjaman tetap 200 (diuji manual).
Statelessness	Tidak ada sesi, cookie, maupun tabel sesi. Setiap request membawa JWT; identitas dan peran dibaca dari token terverifikasi, sehingga request baru dilayani tanpa bergantung pada request sebelumnya.
Composability	Proses pengajuan disusun dari kemampuan yang berdiri sendiri: autentikasi, data fasilitas (Katalog), aturan jadwal, dan penyimpanan pengajuan. Peminjaman menyusun Katalog dengan satu panggilan HTTP tanpa mengubah kode Katalog.

Contoh reuse Katalog. Satu layanan Katalog dipakai dua *consumer* berbeda tanpa perubahan: klien memakai GET /fasilitas untuk menampilkan daftar, sedangkan Peminjaman memakai GET /fasilitas/{id} untuk memvalidasi pengajuan. Layanan baru (misalnya pelaporan kerusakan) dapat memakai endpoint yang sama.

Logic agnostic dan non-agnostic. *Agnostic* (bebas dari proses bisnis tertentu, dapat dipakai ulang): data dan status aktif fasilitas, autentikasi dan verifikasi JWT, pencatatan log request. *Non-agnostic* (khusus proses peminjaman): aturan bentrok jadwal, transisi status *pending* ke *approved*/*rejected*, aturan bahwa hanya mahasiswa yang mengajukan, dan aturan kepemilikan pengajuan.

Bukti request/response antar layanan. Log kedua layanan pada satu pengajuan; token yang sama (***7VGLms) muncul di Peminjaman dan Katalog, dan Katalog memverifikasinya sendiri:

```
[Peminjaman -> Katalog] GET http://localhost:3001/api/v1/fasilitas/1 -> 200 (61ms)
Authorization: Bearer ***7VGLms
[Katalog] GET /api/v1/fasilitas/1 -> 200 (1ms) auth=Bearer ***7VGLms
user=2(mahasiswa)
[Peminjaman] POST /api/v1/peminjaman -> 201 (82ms) auth=Bearer ***7VGLms
user=2(mahasiswa)
```

3. Perancangan dan Implementasi REST API

Resource dan data utama. /fasilitas (Katalog, hanya baca): id, name, category, description, location, capacity, is_active. /peminjaman (Peminjaman): id, facility_id, borrower_id, start_at, end_at, purpose, status, admin_note, processed_by, created_at, updated_at. Nilai borrower_id berasal dari identitas token, bukan dari body. Rincian tujuh endpoint, tipe data, dan validasi ada di *Panduan_dan_Dokumentasi_API.md*.

Alur perubahan status (PATCH /peminjaman/{id}): token (401), peran admin (403), validasi status (400), pengajuan ada (404), status saat ini harus *pending* (409), lalu UPDATE ... WHERE status='pending' yang juga mengisi processed_by dan admin_note. Status akhir tidak dapat diubah.

Method	**Dipakai untuk**	**Status sukses**	**Alasan pemilihan**
GET	Membaca fasilitas dan pengajuan	200	Aman dan idempoten; handler hanya menjalankan SELECT.
POST	Mem ET tidak mengubah data. Skenario U02 memanggil GET /fasilitas dua kali dan hasilnya identik dengan data awal; di kode, handler GET tidak memiliki perintah tulis. buat pengajuan	201 + Location	Membuat resource baru yang ID-nya ditentukan server.
PATCH	Admin mengubah status	200	Hanya sebagian field (status, catatan) yang boleh berubah.

Bukti Richardson Maturity Model Level 2. Level 1: setiap konsep adalah resource ber-URI sendiri (/fasilitas/{id}, /peminjaman/{id}), bukan satu endpoint untuk semua. Level 2: operasi dibedakan dengan verba HTTP (GET, POST, PATCH) dan hasilnya dengan kode

status bermakna (201, 400, 401, 403, 404, 409). Sistem ini belum Level 3 karena respons tidak memuat tautan (HATEOAS).

G

Risiko pengulangan POST. POST tidak idempoten: klik ganda atau *retry* setelah *timeout* dapat membuat pengajuan ganda. Pada sistem ini permintaan identik yang diulang ditolak 409 karena slot sudah diisi pengajuan *pending*, tetapi itu perlindungan tidak langsung, dan klien yang gagal menerima respons pertama akan membaca 409 padahal pengajuannya sudah tersimpan. Kunci idempotensi (Idempotency-Key) tidak diimplementasikan (pengayaan).

PUT dan PATCH. PUT mengganti seluruh representasi resource (idempoten; field yang tidak dikirim dapat hilang), sedangkan PATCH mengubah sebagian. PATCH dipilih karena admin hanya boleh mengubah status dan catatan, sedangkan facility_id, borrower_id, dan waktu tidak boleh berubah. Endpoint PUT tidak dibuat.

4. Otentikasi dan Otorisasi

Alur login. Klien mengirim POST /auth/login. Server mencari akun, membandingkan password dengan *hash* bcrypt, lalu menerbitkan JWT HS256 berisi sub (ID), username, role, iat, dan exp (1 jam). Kegagalan dijawab 401 dengan pesan yang sama untuk akun tidak ada maupun password salah. Peran ditentukan oleh data awal di server, bukan oleh klien.

Bagian JWT. *Header* (algoritma dan tipe), *payload* (klaim di atas), dan *signature* (HMAC-SHA256 atas header dan payload memakai JWT_SECRET dari .env). Payload hanya di-encode base64url, bukan dienkripsi, sehingga siapa pun dapat membacanya; karena itu password tidak dimasukkan. Signature menjamin keutuhan, bukan kerahasiaan.

Verifikasi dan pengiriman token. Klien mengirim Authorization: Bearer <token> pada setiap request terlindungi. Middleware authenticate memanggil jwt.verify dengan algoritma yang ditetapkan server (HS256, bukan dari header token), sehingga signature dan exp diperiksa dan token alg:none ditolak. Token hilang, rusak, kedaluwarsa, atau dimodifikasi dijawab 401. Setelah itu middleware requireRole dan pemeriksaan pemilik di handler menghasilkan 403 bila tidak sesuai.

Operasi	**Mahasiswa**	**Admin**
Melihat katalog dan detail fasilitas	Ya	Ya

Operasi	Mahasiswa	Admin
Mengajukan peminjaman	Ya, atas nama sendiri (borrower_id dari token)	Tidak (403)
Melihat daftar atau detail pengajuan	Hanya milik sendiri (milik orang lain 403)	Seluruh pengajuan
Menyetujui atau menolak pengajuan *pending*	Tidak (403)	Ya

Statelessness. Server tidak menyimpan sesi; keabsahan request ditentukan hanya oleh token dan *secret*, jadi urutan request tidak berpengaruh dan layanan mudah diskalakan. Konsekuensinya token tidak dapat dicabut sebelum exp.

Risiko token dicuri. Pencuri dapat bertindak sebagai pemilik sampai token kedaluwarsa. Mitigasi yang ada: masa berlaku pendek (1 jam), secret hanya di .env, password tidak ada di payload, dan log hanya menampilkan 6 karakter terakhir token. Mitigasi yang disarankan tetapi belum dibuat: HTTPS (wajib di luar localhost; saat ini HTTP lokal), *refresh token* dengan akses lebih pendek, daftar token yang dicabut, dan pemakaian kunci asimetris (RS256) agar kebocoran satu layanan tidak memungkinkan pemalsuan token, karena HS256 memakai secret yang sama di kedua layanan.

5. Hasil Pengujian

Sepuluh skenario dijalankan pada 27 Oktober 2026 dengan skrip uji otomatis (pengujian/run_tests.js, klien HTTP Node.js) terhadap kedua layanan di localhost, dengan data di-reset sebelum uji. Akun: *admin*, *mhs_budi*, *mhs_citra*. Method, URL, header, dan body merujuk Panduan API. Token disamarkan.

ID	Prasyarat / akun	Hasil diharapkan	Status dan respons aktual	Hasil
U01	mhs_budi; password benar lalu salah	200 + JWT; salah: 401 tanpa JWT	200, JWT 3 bagian, tanpa password; salah: 401 *INVALID_CREDENTIALS*	Lulus
U02	mhs_budi; data awal Katalog	200; sesuai data awal; tidak berubah	200 (4 fasilitas sesuai seed); GET kedua identik; detail #1 200	Lulus

ID	Prasyarat / akun	Hasil diharapkan	Status dan respons aktual	Hasil
U03	mhs_budi; fasilitas #1 aktif	201 *pending*; pemilik dari token; Katalog dipanggil	201, *pending*, borrower_id=2; fasilitas #4 409 *FACILITY_INACTIVE*, #99 404 (hanya Katalog yang tahu)	Lulus
U04	admin; dua pengajuan *pending* (Budi, Citra)	200 approved/rejected; terlihat pemilik	200 *approved*, 200 *rejected*, pemilik melihat status; ubah lagi 409 *INVALID_TRANSITION*	Lulus
U05	Budi, Citra; #1 *approved*	400; 409; tidak ada pengajuan baru	400 (end sebelum start); 409 *SCHEDULE_CONFLICT*; jumlah 2 ke 2; bertemu di batas 201	Lulus
U06	tanpa token	401; data tidak berubah	GET, POST, PATCH 401; Katalog 401; jumlah 4 ke 4	Lulus
U07	token signature benar, exp lampau	401; exp diperiksa	401 "Token telah kedaluwarsa"; kontrol exp masa depan 200	Lulus
U08	token Budi, role diubah ke admin tanpa signature baru	401	GET 401, PATCH 401, token *alg:none* 401	Lulus
U09	Budi dan Citra; pengajuan *pending*	403; status tidak berubah	403 (pemilik), 403 (mahasiswa lain); status tetap *pending*	Lulus
U10	Budi vs pengajuan #2 milik Citra (ada)	403; pemilik dari token	403 baca, 403 ubah; POST dengan borrower_id Citra 201 dengan borrower_id=2	Lulus

Hasil gagal dan keterbatasan. Seluruh sepuluh skenario lulus. Keterbatasan yang masih ada: pengujian hanya di localhost dengan HTTP (belum HTTPS); skenario Katalog

dihentikan (503) dan permintaan bersamaan (1 dari 4 POST ke slot yang sama lolos) diuji manual, di luar sepuluh skenario; tidak ada uji beban maupun penanganan *timeout* terperinci; waktu hanya diterima dalam UTC sehingga klien harus mengonversi dari WIB/WITA; pengajuan dengan waktu mulai di masa lalu belum ditolak; dan token tidak dapat dicabut.

6. Evaluasi dan Kontribusi Anggota

Capaian. Dua layanan berjalan sebagai proses terpisah (port 3001 dan 3002) dengan database SQLite masing-masing; tujuh endpoint sesuai kontrak; JWT dengan verifikasi signature dan exp; otorisasi peran dan kepemilikan; klien terminal; log HTTP sebagai bukti komunikasi antar layanan; serta 10 dari 10 skenario uji lulus.

Kendala dan perbaikan. (1) Pemeriksaan jadwal lalu penyimpanan dapat ditembus dua permintaan bersamaan; diperbaiki dengan menjalankan keduanya dalam satu transaksi sinkron. (2) Ketergantungan Peminjaman pada Katalog; kini Katalog yang mati menghasilkan 503, bukan galat server. (3) Pengajuan milik orang lain yang ID-nya ada dijawab 403 (bukan 404) agar sesuai uji kepemilikan U10. [tambahkan kendala yang benar-benar dialami kelompok]. **Usulan perbaikan:** HTTPS, *refresh token*, kunci asimetris RS256, kunci idempotensi pada POST, penolakan waktu di masa lalu, endpoint pengelolaan katalog, pembatalan peminjaman, dan pembatasan percobaan login.

Kontribusi anggota. Tabel berikut diisi kelompok sesuai pekerjaan yang benar-benar dilakukan masing-masing anggota (hapus baris yang tidak dipakai).

Nama / NIM	Pekerjaan konkret	Lokasi berkas / commit	Pengujian
Marchello Lengkong 240211060045	Membuat layanan Katalog, data awal 4 fasilitas (1 nonaktif), endpoint GET /facilities dan GET /facilities/{id}, middleware JWT di Katalog; menulis Bab 2 dan spesifikasi endpoint Katalog	source_code/katalog/; commit 68541e4e60b1dcb755386c5e572ee14c4a5f8291	U02; ikut U03 (verifikasi log di Katalog)

Marcho Roring 240211060072	Membuat layanan Peminjaman: validasi waktu, pemanggilan HTTP ke Katalog, logika overlap, transisi status PATCH; menulis use case dan Bab 4	source_code/peminjaman/; commit 739c214e7be75c6901f3e1f2efeb8ad13abc2dcc	U03, U04, U05
Miracle Sundah 240211060074	Membuat modul login, hashing bcrypt, penerbitan JWT, middleware peran dan kepemilikan, .env.example, data awal akun; menulis Bab 5	source_code/katalog/auth/; commit de471ca8abeb64bfb87e2222c874e5baa5dc5466	U01, U06, U07, U08, U09, U10
Rehan Ibrahim 240211060078	Membuat skrip klien terminal, skrip reset data, Panduan_dan_Dokumentasi_API.md, diagram komponen; mengompilasi laporan dan menulis Bab 1, 3, 6, 7	source_code/klien/, 21de29e45c39daf07f039bb514089944e7474381, f5262dfdf25c39d9fc36024edd8ddf0580078f17, 21f44cd8f27a7036c78e678461fdc599305ed11b	Skenario end-to-end dan latihan demo; kompilasi hasil U01-U10

Pelajaran yand didapat:

Marchello : Nama resource dan path endpoint perlu disepakati dan didokumentasikan sejak awal antar anggota tim, karena ketika penamaan di Katalog (`/fasilitas`) tidak selaras dengan asumsi awal, bagian lain yang bergantung padanya ikut perlu disesuaikan. Membuat log request di setiap layanan sangat membantu membuktikan bahwa sistem benar-benar berjalan sesuai rancangan, bukan hanya sesuai dugaan di kode.

Marcho : Setelah membuat layanan Peminjaman, saya mempelajari bahwa sebuah layanan yang bergantung pada layanan lain harus menangani kondisi saat layanan tersebut tidak tersedia atau lambat merespons, bukan hanya kondisi saat semuanya berjalan lancar.

Miracle : Memverifikasi token ternyata tidak cukup hanya dengan membaca isi payload-nya, karena signature harus benar-benar diperiksa ulang di server menggunakan secret yang sama; satu karakter berbeda pada `JWT_SECRET` antar layanan saja sudah cukup membuat seluruh token ditolak.

Rehan : Saya banyak belajar tentang mendokumentasikan langkah instalasi dan konfigurasi lingkungan secara rinci sangat membantu ketika proyek dipindahkan atau di-*clone* ulang oleh anggota lain, karena hal-hal yang terasa jelas bagi satu orang bisa terlewat bagi orang lain.

Daftar Pustaka

[1] Erl, T. (2007). *SOA: Principles of Service Design*. Prentice Hall.

[2] Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. University of California, Irvine.

[3] Fowler, M. (2010). *Richardson Maturity Model*. martinfowler.com.

[4] Fielding, R., Nottingham, M., & Reschke, J. (2022). *RFC 9110: HTTP Semantics*. IETF.

[5] Dusseault, L., & Snell, J. (2010). *RFC 5789: PATCH Method for HTTP*. IETF.

[6] Jones, M., Bradley, J., & Sakimura, N. (2015). *RFC 7519: JSON Web Token (JWT)*. IETF.

[7] OWASP Cheat Sheet Series. *JSON Web Token for Java Cheat Sheet*. Owasp.org.

Sumber Kode/Bantuan

Node.js; Express 5 (kerangka web); jsonwebtoken (penerbitan dan verifikasi JWT); bcryptjs (hash password); better-sqlite3 (SQLite); dotenv (konfigurasi lingkungan). Semuanya dipasang dari npm sesuai package.json tiap layanan.